



南開大學
Nankai University

南 開 大 學

計 算 機 學 院

並行程序設計實驗報告

ANN SIMD 編程實驗

王 燁

年級：2024 級

專業：計算機科學與技術

指導教師：王剛

2026 年 5 月 9 日

目录

一、 问题描述	1
二、 实验环境	1
三、 实验流程	2
(一) 前期准备	2
(二) Flat-SIMD	2
1. 实现原理	2
2. 汇编语言分析	3
3. 基于 perf 的性能剖析	4
(三) SQ-SIMD	5
1. 实现原理	5
2. 关于 recall 与 latency 的 trade-off	6
3. 基于 perf 的性能剖析	7
(四) PQ-SIMD	8
1. 实现原理	8
2. 关于 recall 与 latency 的 trade-off	9
3. 进一步的优化	10
4. 基于 perf 的性能剖析	13
四、 总结	14

一、问题描述

最近邻搜索 (NNS) 的主要目标为在高维空间中识别与查询向量最接近的向量。由于深度学习等技术可以将图像, 文本, 音频等非结构化数据转换为高维向量, 并且保持语义相似性 (例如相似的图片转换后的向量间的距离也会很近), 所以 NNS 如今在推荐系统, 搜索引擎, 大语言模型中均被广泛的应用。然而对于目前的应用, 向量规模往往大于千万级别, 并且向量的维度通常也大于 100 维, 普通的精确搜索算法的复杂度太高, 很难在实际场景中应用。现在的向量搜索算法通常使用近似最近邻搜索 (ANNS), ANNS 旨在保持较高准确率的同时降低计算时间。

给定一个 d 维向量数据集 $X = \{x_1, x_2, x_3, \dots, x_N\} \subset \mathbb{R}^d$, 用 $\delta(x, y)$ 表示向量 x 和向量 y 的距离。给定一个查询 $q \in \mathbb{R}^d$ 。kNNS 目的在于找到集合 S , S 满足 $|S| = k, S \subseteq X$ 且

$$\forall x \in S, \forall y \notin S, \delta(x, q) < \delta(y, q)$$

直观理解: 集合 S 是 X 中距离 q 最近的前 k 个向量组成的集合。

而 ANNS 并不要求集合 S 中的 k 个向量都是真实最近的 k 个向量, 只要求能达到一定的准确度即可。我们通常使用查询延时 (latency) 和 $\text{recall}@k$ 来衡量 ANNS 算法的效率。其中 $\text{recall}@k$ 定义为

$$\text{recall}@k = \frac{|\text{KNN}(q) \cap \text{KANN}(q)|}{k}$$

其中 $\text{KNN}(q)$ 表示查询的精确答案集合, $\text{KANN}(q)$ 表示使用 ANNS 找到的答案集合。对于 ANNS 算法, 我们在比较加速比时通常选择比较达到某一精度时不同算法的延时 (例如达到 $\text{recall}@100 = 0.95$ 时的延时)。

ANNS 算法的优化可以分为两个方面:

1. 对距离计算速度的优化, 即加快 $\delta(x, y)$ 的计算速度,
2. 减少需要访问点的数量, 普通的算法对于每一个查询, 需要遍历 X 中的每一个点。而一个好的 ANNS 算法会尽可能减少访问点的数量。

在 SIMD 实验中, 我们重点关注对距离计算速度的优化。一般来说, $\delta(x, y)$ 使用欧几里得距离和内积距离。其中欧几里得距离定义为:

$$\delta(x, y) = \sum_{i=1}^d (x_i - y_i)^2$$

内积距离定义为:

$$\delta(x, y) = 1 - \sum_{i=1}^d x_i * y_i$$

朴素的 SIMD 优化是对上述的计算公式用 SIMD 指令进行加速。

一个更好的对距离计算进行优化的算法为 Vector Quantization (VQ)。其中 VQ 又分为 Scalar Quantization (SQ) 和 Product Quantization (PQ)。

二、实验环境

本次实验基于华为鲲鹏服务器开展, 使用 ARM aarch64 架构和 neon 指令集, 所有程序完成及编译运行基于此环境进行, 编译指令为 `g++ main.cc -o main -O2 -fopenmp -lpthread -std=c++11`。

三、 实验流程

(一) 前期准备

由于实验基于 neon 指令集, neon 是 128 位 SIMD 扩展架构, 包含 16 个 128 位寄存器, 我们可以首先对 SIMD 指令封装为 simd8float32.h, 实现封装 float32x4x2_t 并重载加减乘法、批量加载与存储以及水平求和等操作, 这样我们便不再需要关注底层架构, 只需要将计算表达成向量化的形式即可。

(二) Flat-SIMD

1. 实现原理

内积的计算公式 (这里假设下标从 0 开始) 为:

$$\delta(x, y) = 1 - \sum_{i=0}^{d-1} x_i * y_i$$

将内积的计算向量化实际非常简单, 我们将公式写为:

$$\delta(x, y) = 1 - \sum_{i=0}^{[(d-1)/8]} \sum_{j=8i}^{8i+7} x_i * y_i$$

在本实验中, 我们用 DEEP100K 数据集进行测试, 其中数据集中的向量类型为图片, 维度固定为 96。这为我们执行计算向量化提供了天然条件。

常规的串行算法通过暴力搜索, 遍历所有的基向量, 对每一个向量计算内积, 对向量的每一个维度进行遍历, 通过标量乘法计算并累加到相应的 dis, 将内积转换为 dis, 最后通过最大堆执行维护, 堆顶是当前 k 个最近距离中的最大值; 新距离小于堆顶时替换。对于每一个向量, 执行了 96 次标量乘法和标量加法, 对于 query[d] 的访问又重复加载了 96 次基向量, 这大幅限制了算法的性能。

在 SIMD 优化算法中, 我们针对 96 维向量进行了特殊处理。首先将 96 维查询向量按 4 个浮点数一组, 分别加载到 24 个 float32x4_t 中, 这样有效避免了在 query 访问中对向量的重复加载, 因为查询向量在寄存器的值是不变的。

随后, 每次从基向量中连续加载 4 个浮点数, 与对应的查询寄存器 (q0-q23) 进行向量乘法, 得到 4 个元素的乘积向量, 共计 24 次, 再对产生的 24 个向量乘积的所有元素采用成对相加、层层归并的策略累加为一个标量内积。

Algorithm 1 Flat-SIMD

Input: 基向量矩阵 $B_{N \times d}$, 查询向量 $q \in \mathbb{R}^d$, 返回数目 k , 且 $d = 96$

Output: 距离最小的 k 个索引及距离

- 1: 将 q 按每 4 个浮点数一组加载至 24 个 NEON 寄存器 $qreg_0, \dots, qreg_{23}$
- 2: 创建空的堆 Q (容量 k), 按距离降序排列
- 3: **for** $i \leftarrow 0$ **to** $N - 1$ **do**
- 4: $b \leftarrow B[i, :]$ ▷ 当前基向量指针
- 5: **if** $i + 3 < N$ **then**
- 6: $__\text{builtin_prefetch}(B[i + 3, :])$ ▷ 软件预取
- 7: **end if**
- 8: // 24 路向量乘法, 每路处理 4 个浮点数
- 9: **for** $j \leftarrow 0$ **to** 23 **do**

```

10:      $p_j \leftarrow \text{vmulq\_f32}(\text{vld1q\_f32}(\mathbf{b} + 4j), q_{reg_j})$ 
11: end for
12: // 树形归约求和
13:  $s_{0..11} \leftarrow p_{2m} + p_{2m+1}, m = 0 \dots 11$ 
14:  $s_{0..5}^{(1)} \leftarrow s_{2m} + s_{2m+1}, m = 0 \dots 5$ 
15: ... (继续合并至单个 4 元素向量)
16:  $sum \leftarrow \text{horizontal\_add}(s_{final})$  ▷ 标量内积
17:  $dis \leftarrow 1.0 - sum$ 
18: if  $Q.size() < k$  then
19:      $Q.push(\{dis, i\})$ 
20: else if  $dis < Q.top().first$  then
21:      $Q.pop(), Q.push(\{dis, i\})$ 
22: end if
23: end for
24: return  $Q$ 

```

表 1: 性能对比

算法	Recall (召回率)	latency(μs)	加速比
Flat_scan	0.99995	20164.0	1.00 $\times$
Flat_simd	0.99995	4196.8	4.80 $\times$

在维持召回率一致的情况下，我们的 SIMD 算法实现了高效的性能优化。

2. 汇编语言分析

通过 Compiler Explorer, 我们对 flat_scan.h 和 flat_simd.h 进行汇编语言层面的分析, 分析环境与实验配置一致, 为 ARM64 GCC 10.3.0, -O2 -march=armv8-a -fverbose-asm.

通过汇编语言, 我们可以发现, 在 flat_scan 算法中, 针对主循环 L37, flat_scan 每次执行 6 次标量指令, 针对 96 维向量, 连同内层循环, 共计执行 576 条指令, 而在 flat_simd 算法中, 针对主循环 L34, 我们完全执行 NEON 向量指令, 同时对 96 个 float 并行处理, 计算部分位于 L38 到水平归约部分, 共执行了 18 次基向量加载, 24 次查询向量加载, 24 次向量乘法 fmul, 23 次归约树加法 (1 次 fmla 和 23 次 fadd) 和 4 次水平归约 (1 次 dup 和 1 次 faddp 以及 2 条非向量指令), 核心指令数为 91 条, 且消除了内层循环, 大幅削减了指令执行数, 提升了 CPU 流水线效率, 同时由于单次抓取 4 个浮点, 内存访问量也会大幅减少。

```

.L37:
    ldr s2, [x19, x5] // MEM[base: _228, index: ivtmp.205_98, offset: 0B], MEM[base: _228, index: ivtmp.205_98, offset: 0B]
    ldr s1, [x21, x5] // MEM[base: query_27(0), index: ivtmp.205_98, offset: 0B], MEM[base: query_27(0), index: ivtmp.205_98, offset: 0B]
    add x5, x5, 4 // ivtmp.205, ivtmp.205
    fmadd s0, s2, s1, s0 // dis, MEM[base: _228, index: ivtmp.205_98, offset: 0B], MEM[base: query_27(0), index: ivtmp.205_98, offset: 0B]
    cmp x20, x5 // _202, ivtmp.205
    bne .L37 //,
    sub x2, x1, x0 // tmp186, pretmp_233, pretmp_231
    fsub s0, s8, s0 // _243, _243, dis
    cmp x26, x2, asr 3 // k, tmp186,
    bhi .L73 //,

```

图 1: flat_scan

```
.L54:
add x0, x21, #3 // tmp391, i,
sub x6, x19, #368 // _462, ivtmp.214,
cmp x22, x0 // base_number, tmp391
bls .L38 //,
prfm PLDL1KEEP, [x25, x19] /* ivtmp.214
```

图 2: flat_simd part1

```
.L38:
sub x5, x19, #352 // tmp394, ivtmp.214,
sub x4, x19, #336 // tmp395, ivtmp.214,
ldr q3, [sp, 448] // _115, %sfp
sub x3, x19, #320 // tmp396, ivtmp.214,
ldr q19, [x5] // _91,
sub x2, x19, #304 // tmp397, ivtmp.214,
ldr q18, [x6] // _92, *_462
sub x1, x19, #288 // tmp398, ivtmp.214,
fmul v19.4s, v19.4s, v3.4s // _88, _91, _115
sub x0, x19, #272 // tmp399, ivtmp.214,
ldr q3, [sp, 464] // _117, %sfp
ldr q16, [x4] // _89,
fmul v18.4s, v18.4s, v3.4s // _90, _92, _117
ldr q3, [sp, 432] // _114, %sfp
ldp q4, q21, [x19, -256] // _79, _77,
fmul v16.4s, v16.4s, v3.4s // _86, _89, _114
ldr q3, [sp, 352] // _109, %sfp
ldp q17, q20, [x19, -224] // _75, _73,
fmul v4.4s, v4.4s, v3.4s // _76, _79, _109
ldr q3, [sp, 336] // _108, %sfp
ldp q5, q2, [x19, -192] // _71, _69,
fmul v21.4s, v21.4s, v3.4s // _74, _77, _108
ldr q3, [sp, 320] // _107, %sfp
ldp q0, q1, [x19, -160] // _67, _127,
fmul v17.4s, v17.4s, v3.4s // _72, _75, _107
```

图 3: flat_simd part2

3. 基于 perf 的性能剖析

在相同框架下，我们通过 perf 对程序的缓存性能、指令执行以及执行时间等进行剖析，以此与我们的汇编分析交叉印证。

在服务器中对 flat_scan.h 和 flat_simd.h 版本的 main.cc 程序执行：perf stat -e cycles, instructions, cache-misses, cache-references ./main¹，结果如下：

上述数据有力印证了前文对汇编代码的分析结论：

- 串行指令数 1182.7 亿，SIMD 降至 222.7 亿，缩减 81.2%。这与前文的汇编指令数对比分析结果一致。
- SIMD 版本缓存引用数为 102 亿，远小于串行的 388 亿，说明内存访问量大幅减少，这正是因为向量指令所带来的单次调用量提升。

¹由于服务器负载因素，实际测试性能会出现波动，对于每次测试结果，结果仅适用于当前负载环境，但是理论分析不受影响。

表 2: flat_scan VS flat_simd

性能指标	flat_scan	flat_simd
总执行时间 (s)	33.56	8.26
用户时间 (s)	33.42	8.20
系统时间 (s)	0.024	0.032
CPU 周期数	86,234,056,354	21,037,172,583
指令数	118,270,998,865	22,271,090,904
IPC (指令/周期)	1.37	1.06
缓存引用	38,834,202,108	10,233,781,803
缓存缺失	493,570,981	153,565,216
缓存缺失率	1.27%	1.50%
加速比	-	×4.06
指令减少比例	-	81.2%

(三) SQ-SIMD

在 Flat 方法的基础上, 可以进一步通过 SQ 来减少计算与访存开销。

1. 实现原理

由于原向量一般每一维度都用一个 32bit 的浮点数组存储, 假如我们使用 128bit 的 SIMD 寄存器加速计算, 最多只能一次算 4 个浮点数。但是 SQ 可以将向量的每一个维度量化为 8bit 的无符号整型, 此时我们再用 128bit 的寄存器加速计算时, 可以一次算 16 个 uint8。

对于每个维度 i , 将 x_i 映射到 $[0, 255]$ 范围内:

$$x'_i = \text{round} \left(255 \cdot \frac{x_i - \min_i}{\max_i - \min_i} \right)$$

其中 $\min_i$ 和 $\max_i$ 是第 i 维的最小值和最大值。

随后使用量化后的整数值近似计算距离, 结合 SIMD 指令进一步提升效率。

基于 SQ 的检索通常采用两阶段流程: 首先在量化空间中进行粗排, 快速筛选出 Top-p 个候选向量; 随后在原始空间中对这 p 个候选进行精排 (rerank), 得到最终的 Top-k 结果。由于量化会引入误差, 精排是保证 recall 的必要步骤。

在 SIMD 优化方面, 粗排阶段的距离计算仍然可以复用 Flat-SIMD 中的向量化方法, 同时可以结合更紧凑的数据表示提升访存效率; 精排阶段本质上仍是原始向量的距离计算, 可以直接复用 Flat-SIMD 实现。

基于上述分析, 我们编写了 sq_simd.h, 相较于 flat_simd.h, 做出如下关键优化:

Algorithm 2 SQ-SIMD

Input: 基向量 $\mathbf{B}_{N \times d}$, 量化编码 $\mathbf{C}_{N \times d}$ (uint8), 查询 $\mathbf{q}$, 参数 mins, scales; 返回数 k , 粗排候选数 p

Output: 最终 top-k 结果

- 1: // **离线部分:** 由 $\mathbf{B}$ 构建 mins, scales, 并量化至 $\mathbf{C}$ (未画出)
- 2: // **在线查询**
- 3: 预计算 $\mathbf{q}_{scaled}[j] \leftarrow \mathbf{q}[j] \cdot \text{scales}[j], \forall j < d$
- 4: 将 $\mathbf{q}_{scaled}$ 每 4 个元素加载至 NEON 寄存器数组 $qreg_0, \dots, qreg_{d/4-1}$
- 5: 初始化候选缓冲区 buf_d (距离), buf_i (索引), 容量 p , 全空

```

6: for  $i \leftarrow 0$  to  $N - 1$  do
7:   if  $i + 4 < N$  then
8:     __builtin_prefetch( $C[i + 4, :]$ )
9:   end if
10:   $s_0, \dots, s_3 \leftarrow 0$  ▷ 4 个 128 位累加器
11:  for  $t \leftarrow 0$  to  $d/4 - 1$  step 4 do
12:     $c \leftarrow \text{vld1q\_u8}(C[i, 4t])$  ▷ 一次加载 16 个 uint8
13:    将  $c$  扩展并转换为 4 个 float32x4_t:  $f_0, \dots, f_3$ 
14:     $s_0 \leftarrow \text{vmlaq\_f32}(s_0, f_0, \text{qreg}_{t+0})$ 
15:     $s_1 \leftarrow \text{vmlaq\_f32}(s_1, f_1, \text{qreg}_{t+1})$ 
16:     $s_2 \leftarrow \text{vmlaq\_f32}(s_2, f_2, \text{qreg}_{t+2})$ 
17:     $s_3 \leftarrow \text{vmlaq\_f32}(s_3, f_3, \text{qreg}_{t+3})$ 
18:  end for
19:  归约  $s_0 + s_1 + s_2 + s_3$  得估计内积  $ip$ ,  $dist \leftarrow 1 - ip$ 
20:  用  $dist$  更新  $buf$  中最大的  $p$  个元素 (线性扫描替换最差值)
21: end for
22: 将  $buf$  按距离升序排序, 得到候选索引集  $C$ 
23: // 精排阶段
24: 创建空的最大堆  $Q$  (容量  $k$ )
25: for  $idx \in C$  do
26:   调用 flat_inner_product( $B[idx, :]$ ,  $q, d$ ) 得到精确内积
27:    $dis \leftarrow 1 - \text{内积}$ 
28:   if  $Q.size() < k$  then
29:      $Q.push(\{dis, idx\})$ 
30:   else if  $dis < Q.top().first$  then
31:      $Q.pop()$ ,  $Q.push(\{dis, idx\})$ 
32:   end if
33: end for
34: return  $Q$ 

```

2. 关于 recall 与 latency 的 trade-off

在 SQ-SIMD 中, 参数 p 控制粗排向精排传递的候选向量数量。粗排阶段扫描全库的 8-bit 量化编码, 用 SIMD 高效估算距离并筛选出最近的 p 个向量, 精排阶段则对这 p 个候选在原始 96 维浮点向量上执行精确内积, 其开销与 p 成线性关系。理论上, p 值越大, 精排准确度越高, 但相应的耗时也会增加, 所以控制 p 的参数大小, 本质是在进行 recall 与 latency 的 trade-off。

表 3: 不同 p 值下 SQ-SIMD 的 recall 与 latency

p	recall	latency (μs)
10	0.962656	3643.96
20	0.99995	3771.31
50	0.99995	4078.09
100	0.99995	4539.53

根据测试数据，注意到 $p=10$ 时召回率为 0.9626，说明在量化编码下部分真实邻居被过早丢弃，量化域与精确域排序偏差明显；而当 $p=20$ 或更高时，其召回率与 flat_simd 一致，保持了高准确度，在 p 的增加过程中延迟逐步升高，甚至超过了 flat_simd，这符合我们对于 SQ-SIMD 的原理理解，此时 $p=20$ 是针对 SQ-SIMD 的最优参数。

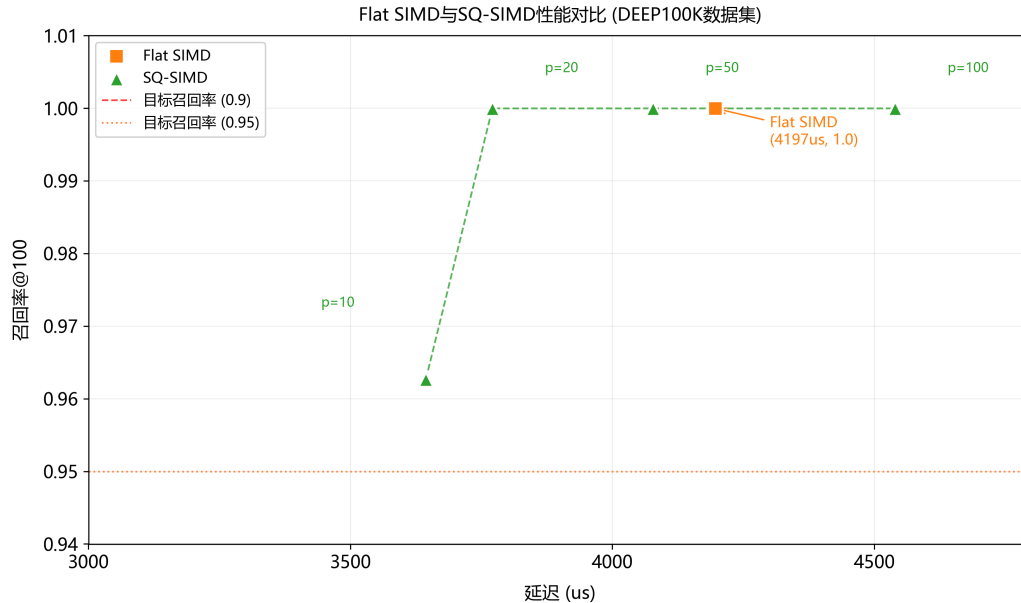


图 4: SQ-SIMD trade-off

3. 基于 perf 的性能剖析

与之前一样，我们在服务器中通过 perf 对程序性能进行剖析：

表 4: flat_simd VS sq_simd ($p=20$)

性能指标	flat_simd	sq_simd ($p=20$)
总执行时间 (s)	8.26	8.03
用户时间 (s)	8.20	7.99
系统时间 (s)	0.032	0.024
CPU 周期数	21,037,172,583	20,410,081,926
指令数	22,271,090,904	29,778,357,892
IPC (指令/周期)	1.06	1.46
缓存引用	10,233,781,803	3,923,278,996
缓存缺失	153,565,216	133,855,233
缓存缺失率	1.50%	3.41%
加速比	-	$\times 1.03$

相比于 Flat_SIMD, SQ-SIMD 在执行时间和 CPU 周期数方面稍有下降，提升了一定性能，而最大的优化在于缓存引用部分：

Flat_SIMD 对每个数据库向量需要加载 96 个 float32，共计 384 字节。循环内通过 24 条 vld1q_f32 指令加载，产生了极高的总访问量，导致 cache-references 过高。

相比之下，SQ-SIMD 通过 unit8 量化编码，每个向量仅 96 字节，内循环每次用 vld1q_u8

加载，一个向量加载 6 次，总访问量大幅下降，cache-references 锐减，明显缓解了内存压力，同时在对 p 的调节下保持了一致的召回率。

(四) PQ-SIMD

PQ 是对 Flat 方法的另一种优化手段，其基本原理与 SQ 一致，同样采用粗排与精排的两阶段检索流程，主要差异在于对粗排阶段的优化加速。

1. 实现原理

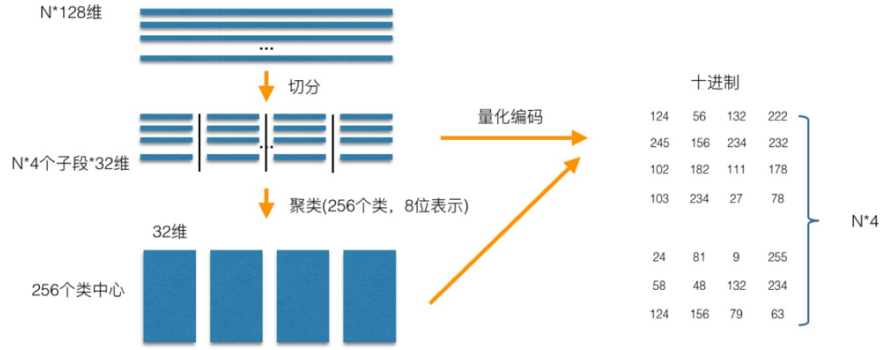


图 5: the example of PQ

PQ 实际利用了分治的基本思想, 将高维向量切分为多个互不相交的低维子段, 利用 K-Means 聚类等手段在每个子段上独立进行向量量化, 编码时, 向量的每个子段被替换为该子段内与其距离最近的聚类中心的 ID, 原始的浮点向量最终被压缩为 M 个整数, 大大提高了压缩效率。

在后续查询阶段, 当一个查询向量到达时, 计算其与经 PQ 压缩后的 Base 向量之间的距离主要有两种策略: 对称距离计算 (SDC) 同时对查询进行量化编码, 利用预先算好的中心间距离表进行查表累加; 非对称距离计算 (ADC) 则保留查询向量的原始精度, 动态计算查询子段到各聚类中心的精确距离生成查找表 (LUT), 再根据数据库编码在 LUT 中查表累加。

Algorithm 3 PQ Search

Input: 查询向量 q , PQ 索引 (M 个子空间, 码本 $C^{(m)}$, 编码 $\{c_{i,m}\}$), 候选数 p , 最终结果数 k

Output: Top- k 最近邻

```

1: // 构建 LUT
2: for  $m = 0$  到  $M - 1$  do
3:    $q^{(m)} \leftarrow$  查询子向量
4:   for  $c = 0$  到  $K_s - 1$  并行 do
5:      $LUT^{(m)}[c] \leftarrow \text{SIMD\_内积}(q^{(m)}, C^{(m)}[c])$  ▷ SIMD 加速
6:   end for
7: end for
8: // 量化空间粗排
9: 维护大小为  $p$  的最大堆  $H$ 
10: for each base vector  $i = 0$  到  $N - 1$  并行 do
11:    $dot \leftarrow 0$ 
12:   for  $m = 0$  到  $M - 1$  do
13:      $dot \leftarrow dot + LUT^{(m)}[codes[i][m]]$  ▷ 标量查表累加

```

```

14:   end for
15:    $dist \leftarrow 1.0 - dot$ 
16:   将  $(dist, i)$  插入堆  $H$  (保持前  $p$  小)
17: end for
18: // 精排
19: 对候选集执行 SIMD 精确内积重排, 返回 Top- $k$ 

```

2. 关于 recall 与 latency 的 trade-off

在 PQ-SIMD 中, 与 SQ-SIMD 关于 p 值的论述一样, p 值作用于精排阶段的精确计算, 所以对 p 值的改动会引发与 SQ-SIMD 类似的, 关于 recall 与 latency 的 trade-off。

表 5: 不同 p 值下 PQ-SIMD 的 recall 与 latency

p	recall	latency (μs)
200	0.606251	385.90
500	0.797052	812.81
1000	0.903354	1647.41
1500	0.942904	2316.34
2000	0.962903	3036.53

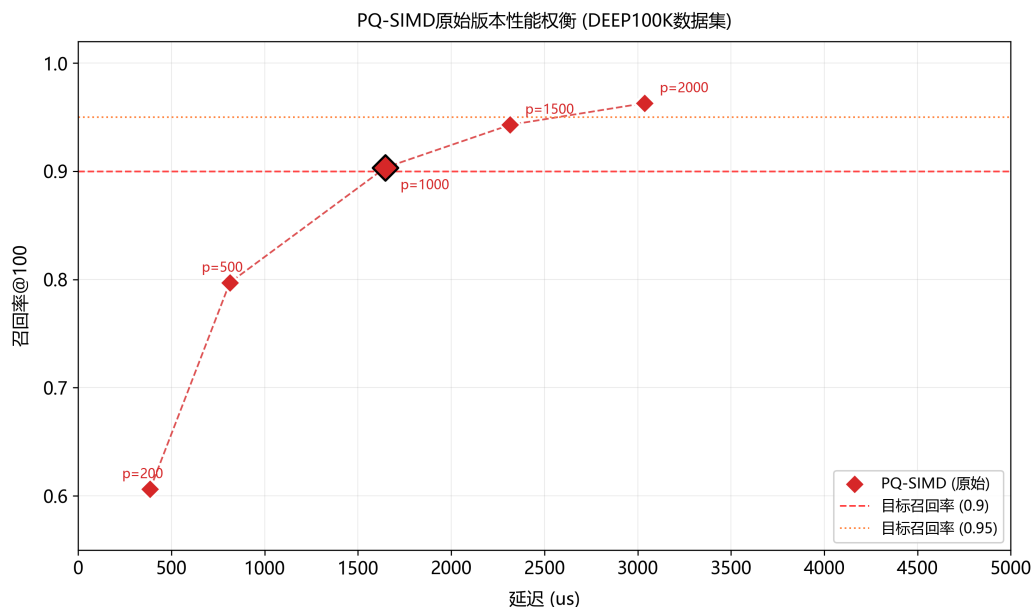


图 6: PQ-SIMD trade-off

注意到相对于 SQ-SIMD, p 值的选取区间及分度扩大, 这其实是符合我们的算法原理的: SQ 的粗排直接在 96 维量化空间内计算完整的量化内积, 保留了全部维度的信息, 距离近似与真实内积的相关性较高, 而 PQ 将 96 维向量切分为 4 段, 每段用聚类中心索引替代, 粗排时仅通过 4 次查表累加来估算距离, 相当于在 4 个独立子空间内做局部的、更粗糙的近似, 丢弃了段内维度的相对几何信息, 量化误差远大于 SQ, 因此我们需要更大的 p 对精排阶段执行召回率回归。

当 $p=200$ 时, PQ-SIMD 用较低的 recall 换取了极低的 latency, 随着 p 值的逐渐增大, 在

p=1000 时达到第一召回率标准, p=2000 时达到第二召回率标准, 总体 latency 仍低于 SQ-SIMD 和 Flat-SIMD, 这说明我们的 PQ-SIMD 实现了更好的优化。

3. 进一步的优化

目前的 PQ-SIMD 相较于 SQ-SIMD, 从根本上改变了量化编码逻辑, 当子空间数 $M=4$ 时, 每条向量仅需存储 4 字节索引, 内存占用和访存带宽大幅降低, 同时在粗排阶段引入查表累加, 仅需 4 次查表与加法即可估算距离, 极大减少了在线查询的计算量。

在此基础上, 我们仍可以对 PQ-SIMD 做出进一步的优化:

- 当前 PQ-SIMD 的数据布局为 AoS (结构体数组), 而 AoS 是 SIMD 不友好的, 这是因为对于某一类运算, 往往只需要调取结构体中的部分属性, 此时采用 AoS 会导致大量的冗余数据加载, 同时每次运算只能调取一个数组元素, 无法发挥 SIMD 的流水线并行效率。
- 在 K-Means 聚类的 Lloyd 迭代部分和 LUT 构建的 base 量化编码部分, 使用的是串行调用算法, 采取的是与 flat_scan 相同的双循环遍历, 每次调用一次 simd_dot_product 函数, 而没有通过循环展开和 NEON 向量指令等手段执行 SIMD 优化。
- 在查表累加阶段, 采取的仍然是纯标量指令, 而没有启用 NEON 向量指令等手段进行并行优化, 也没有启用 LUT 量化和寄存器保存等手段避免重复加载。

因此, 针对上述缺陷, 我们对 PQ-SIMD 做出逐步优化:

Algorithm 4 new PQ-SIMD

```

1: SoA 转换 (AoS  $\rightarrow$  SoA):
2: for  $m = 0$  to  $M - 1$  do
3:   for  $d = 0$  to  $subdim - 1$  do
4:     for  $c = 0$  to  $K_s - 1$  do
5:        $codebook\_soa[m][d][c] \leftarrow codebook[m][c][d]$ 
6:     end for
7:   end for
8: end for ▷ AoS 转换为 SoA

9: function SIMD_DOT_8CENTROIDS_SOA( $subvec, centroids\_soa, subdim, c\_start$ )
10:    $acc[0..7] \leftarrow 0$ 
11:   for  $d = 0$  to  $subdim - 1$  do
12:      $sv \leftarrow subvec[d]$ 
13:      $cvals[0..7] \leftarrow centroids\_soa[d][c\_start \dots c\_start + 7]$ 
14:     for  $t = 0$  to 7 do
15:        $acc[t] \leftarrow acc[t] + sv \times cvals[t]$  ▷ SIMD 乘累加
16:     end for
17:   end for
18:   return  $acc[0..7]$ 
19: end function

20: function SIMD_DOT_8CENTROIDS_4SUBVECS_SOA( $subvec[0..3], centroids\_soa, subdim,$ 
     $c\_start$ )
21:    $acc[0..3][0..7] \leftarrow 0$ 

```

```

22:   for  $d = 0$  to  $subdim - 1$  do
23:        $sv_j \leftarrow subvec[j][d]$  for  $j = 0..3$ 
24:        $cvals[0..7] \leftarrow centroids\_soa[d][c_{start} \dots c_{start} + 7]$ 
25:       for  $j = 0$  to 3,  $t = 0$  to 7 do
26:            $acc[j][t] \leftarrow acc[j][t] + sv_j \times cvals[t]$  ▷ SIMD 乘累加
27:       end for
28:   end for
29:   return  $acc[0..3][0..7]$ 
30: end function

31: for  $iter = 0$  to 14 do
32:   分块并行 for:  $block\_start = 0$  step  $BLOCK$  to  $N$ 
33:   for  $i = block\_start$  step 4 to  $\min(block\_start + BLOCK, N)$  do
34:        $count \leftarrow \min(4, N - i)$ 
35:       收集索引  $i$  处的  $subvec[0..count - 1]$ 
36:       初始化  $best\_dot[j] \leftarrow -\infty$ ,  $best\_c[j] \leftarrow 0$  for  $j = 0..count - 1$ 
37:       for  $c = 0$  step 8 to  $K_s$  do
38:           if  $count = 4$  then
39:                $dots[0..3][0..7] \leftarrow$ 
SIMD_DOT_8CENTROIDS_4SUBVECS_SOA( $subvec[0..3]$ ,  $centroids\_soa$ ,  $subdim$ ,  $c$ )
40:           else
41:               for  $j = 0$  to  $count - 1$  do
42:                    $dots[j][0..7] \leftarrow$ 
SIMD_DOT_8CENTROIDS_SOA( $subvec[j]$ ,  $centroids\_soa$ ,  $subdim$ ,  $c$ )
43:               end for
44:           end if
45:           for  $j = 0$  to  $count - 1$ ,  $t = 0$  to 7 do
46:               if  $dots[j][t] > best\_dot[j]$  then
47:                    $best\_dot[j] \leftarrow dots[j][t]$ ,  $best\_c[j] \leftarrow c + t$ 
48:               end if
49:           end for
50:       end for
51:       for  $j = 0$  to  $count - 1$  do
52:            $assign[i + j] \leftarrow best\_c[j]$ 
53:       end for
54:   end for
55: end for ▷ block 化 + 跨 centroid SIMD

56: for  $m = 0$  to  $M - 1$  do
57:    $query\_sub \leftarrow query + m \cdot subdim$ 
58:    $cb\_soa \leftarrow codebook\_soa[m]$ 
59:   并行 for:  $c = 0$  step 8 to  $K_s$ 
60:   LUT[ $m$ ][ $c..c + 7$ ]  $\leftarrow$  SIMD_DOT_8CENTROIDS_SOA( $query\_sub$ ,  $cb\_soa$ ,  $subdim$ ,  $c$ )
61: end for ▷ SoA 8 中心并行

```

▷ 粗排: 4 路 SIMD 加法

```

62: for 每条查询 do
63:   并行 for:  $i = 0$  step 4 to  $N$ 
64:   加载 4 个向量的编码, 提取索引  $idx[0..3][0..3]$ 
65:    $dots\_vec \leftarrow \mathbf{0}$  ▷ NEON float32x4_t 零向量
66:   for  $m = 0$  to  $M - 1$  do
67:      $lut\_vals[0..3] \leftarrow LUT[m][idx[0..3][m]]$  ▷ 标量查表
68:      $dots\_vec \leftarrow dots\_vec + \text{向量加载}(lut\_vals)$  ▷ SIMD 加法
69:   end for
70:   提取距离并更新 top- $p$  堆
71: end for

```

我们将优化后的 PQ-SIMD 封装为 pq_simd2.h, 在服务器中通过控制变量法检验我们的优化效果²:

表 6: PQ-SIMD VS new PQ-SIMD

p	recall		latency (us)	
	PQ-SIMD	new PQ-SIMD	PQ-SIMD	new PQ-SIMD
500	0.797052	0.797252	812.811	875.991
1000	0.903354	0.903103	1647.41	1490.99
1500	0.942904	0.942753	2316.34	2155.13
2000	0.962903	0.962853	3036.53	2865.3

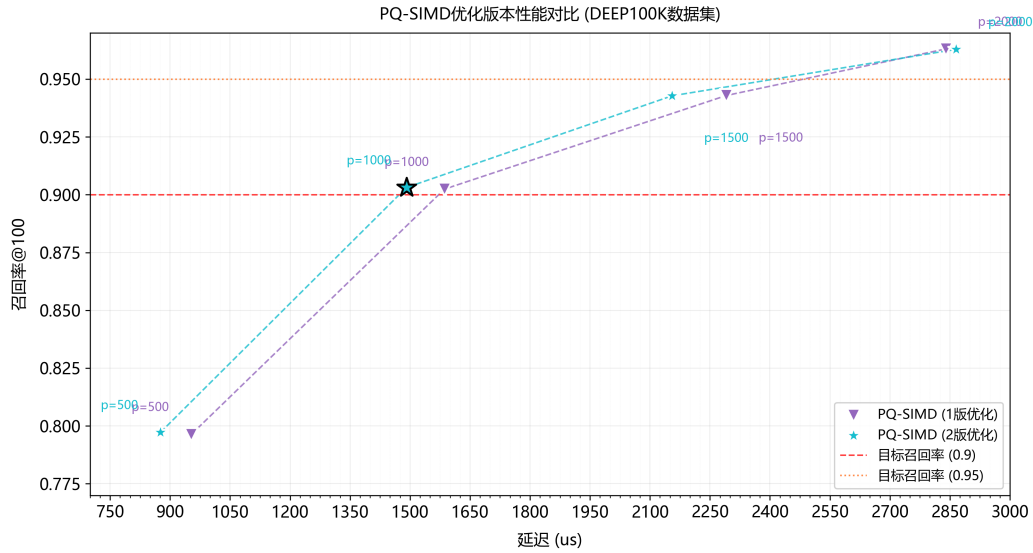


图 7: PQ-SIMD VS new PQ-SIMD

根据测试结果, 我们可以看出, 优化后的 PQ-SIMD 在相同 p 值的情况下, 维持了一致的 recall, 同时 latency 出现了一定的下降。

²绘制 trade-off 对比图时使用的测试数据与上述表格存在差异, 这是由于服务器负载因素导致的性能波动。

现象似乎并不明显，这可能是由于服务器负载因素导致的性能波动，干扰了我们的验证。总之，我们需要通过更精确的性能剖析证明我们的优化。

4. 基于 perf 的性能剖析

我们仍然在服务器中通过 perf 对程序性能进行剖析：

表 7: PQ-SIMD VS new PQ-SIMD (p=1000)

性能指标	PQ-SIMD	new PQ-SIMD
总执行时间 (s)	12.694	8.577
用户时间 (s)	49.944	35.653
系统时间 (s)	10.893	7.957
CPU 周期数	130,098,713,700	92,489,395,808
指令数	322,355,749,776	231,668,981,481
IPC (指令/周期)	2.48	2.50
缓存引用	68,417,881,117	52,946,197,358
缓存缺失	30,126,299	45,634,707
缓存缺失率	0.044%	0.086%
加速比	-	×1.48
指令减少比例	—	28.1%

从 perf 分析结果中，我们可以明显看出新版 PQ-SIMD 的优化效果：

- 总执行时间下降 32.4%：通过跨 centroid 和跨向量并行手段，将 K-Means 分配与 LUT 构建中的重复循环融合为一次处理 8 中心或 4 子向量的操作，大幅降低指令发射数。同时通过将 AoS 调整为 SoA 布局，聚类中心得以按维度连续存放，缓存行利用率提高，减少了内存延迟。
- 指令数下降 28.1%：同样是因为跨 centroid 和跨向量并行手段，将 8 中心或 4 子向量的乘加融合，通过循环展开和 NEON 向量指令压缩了总执行量。
- 缓存引用数降低 22.6%：通过 SoA 布局，同一维度的连续 256 个中心值顺序存放，缓存行容纳量大幅提升，单次加载量提升，大幅减少了 K-Means 分配与 base 编码时对聚类中心数据的重复引用。同时通过 block 化，一个 block 内的全部子向量会在短时间内访问同一组 SoA 聚类中心，减少了缓存行的无效加载。

四、 总结

最后，我们将对于 ANN 算法的 SIMD 优化进行汇总对比：

表 8: ANN 算法综合性能对比（最优值加粗）

算法	recall	延迟 (us)	加速比
Flat (串行)	0.99995	20164	1.00×
Flat (SIMD)	0.99995	4197	4.80×
SQ-SIMD ($p=10$)	0.96266	3644	5.53×
SQ-SIMD ($p=20$)	0.99995	3771	5.35×
SQ-SIMD ($p=50$)	0.99995	4078	4.95×
SQ-SIMD ($p=100$)	0.99995	4540	4.44×
PQ-SIMD ($p=500$)	0.79705	813	24.81×
PQ-SIMD ($p=1000$)	0.90335	1647	12.24×
PQ-SIMD ($p=1500$)	0.94290	2316	8.70×
PQ-SIMD ($p=2000$)	0.96290	3037	6.64×
PQ-SIMD2 ($p=500$)	0.79725	876	23.02×
PQ-SIMD2 ($p=1000$)	0.90310	1491	13.53×
PQ-SIMD2 ($p=1500$)	0.94275	2155	9.36×
PQ-SIMD2 ($p=2000$)	0.96285	2865	7.04×

从表中可以清晰看出：

- 暴力搜索算法（特别是标量实现）虽然提供了精确召回，但其查询延迟也最高。
- 标量化 (SQ) 技术维持了接近暴力搜索的高召回率，同时更进一步降低了延迟。
- 乘积量化方法 (PQ) 相较于 SQ 在召回率方面做出一定取舍，由此换取了极大程度的延迟降低。也相应增加。

总的来说，在本次实验中我们对 ANN 及其优化算法实现了 SIMD 实现，对于 recall 与 latency 的 tradeoff 有一定的了解，在同样使用并行或串行的情况下，我们需要根据需求选择速度和准确率达到要求平衡的方法实现。此外，在本次实验中我们还熟练了有关服务器的使用，对于编译器优化，perf 性能剖析以及汇编语言分析等方面也有了更深入的了解。

附录：AI 使用说明

本次实验中，我使用了 deepseek, 豆包等 AI 工具进行以下辅助分析：

- 代码调试与性能分析讨论
- 实验数据表格的 LaTeX 排版
- 关于实现代码的原理分析与交叉印证

典型对话记录链接如下：

[对话记录 1](#)

[对话记录 2](#)

所有 AI 生成的文字和代码均经过本人充分理解与验证，符合学术诚信。